

ЛАБОРАТОРНАЯ РАБОТА №17 (2+2+3+3 БАЛЛОВ)

ДВУСВЯЗНЫЕ СПИСКИ

Вариант	Задача 1	Задача 2	Задача 3	Задача 4
1	Dynamic31	Dynamic46	Dynamic59	Dynamic70
2	Dynamic32	Dynamic47	Dynamic60	Dynamic71
3	Dynamic33	Dynamic48	Dynamic61	Dynamic72
4	Dynamic34	Dynamic49	Dynamic62	Dynamic73
5	Dynamic35	Dynamic50	Dynamic63	Dynamic74
6	Dynamic36	Dynamic51	Dynamic64	Dynamic75
7	Dynamic37	Dynamic52	Dynamic65	Dynamic76
8	Dynamic38	Dynamic53	Dynamic66	Dynamic77
9	Dynamic39	Dynamic54	Dynamic67	Dynamic78
10	Dynamic40	Dynamic55	Dynamic68	Dynamic79
11	Dynamic41	Dynamic56	Dynamic69	Dynamic80
12	Dynamic42	Dynamic57	Dynamic63	Dynamic71
13	Dynamic43	Dynamic58	Dynamic64	Dynamic74
14	Dynamic44	Dynamic51	Dynamic65	Dynamic78
15	Dynamic45	Dynamic55	Dynamic66	Dynamic75

ДИНАМИЧЕСКИЕ СТРУКТУРЫ ДАННЫХ: ДВУСВЯЗНЫЙ СПИСОК

В заданиях данной подгруппы структура «двусвязный список» (doubly linked list) моделируется цепочкой узлов-объектов типа Node, связанных как с предыдущим, так и с последующим узлом. Свойство Next последнего элемента цепочки и свойство Prev первого элемента цепочки равны null. Для доступа к любому элементу двусвязного списка достаточно иметь ссылку на один из его элементов, однако для ускорения операций со списком удобно хранить три ссылки: на первый элемент списка (first), на его последний элемент (last) и на текущий элемент (current). Для пустого списка все эти ссылки полагаются равными null. Значением элемента списка считается значение его свойства Data.

Dynamic31. Дана ссылка A0 на один из элементов непустого двусвязного списка. Вывести число N — количество элементов в списке, а также ссылки A1 и A2 на первый и последний элементы списка.

Dynamic32. Даны числа D1 и D2 и ссылка A0 на один из элементов непустого двусвязного списка. Добавить в начало списка новый элемент со значением D1, а в конец — новый элемент со значением D2. Вывести ссылки на первый и последний элементы полученного списка.

Dynamic33. Дано число D и ссылка A0 на один из элементов непустого двусвязного списка. Вставить перед данным элементом списка новый элемент со значением D и вывести ссылку на добавленный элемент списка.

Dynamic34. Дано число D и ссылка A0 на один из элементов непустого двусвязного списка. Вставить после данного элемента списка новый элемент со значением D и вывести ссылку на добавленный элемент списка.

Dynamic35. Даны ссылки A1 и A2 на первый и последний элементы двусвязного списка, содержащего не менее двух элементов. Продублировать в списке первый и последний элементы (новые элементы добавлять перед существующими элементами с такими же значениями) и вывести ссылку на первый элемент преобразованного списка.

Dynamic36. Даны ссылки A1 и A2 на первый и последний элементы двусвязного списка, содержащего не менее двух элементов. Продублировать в списке первый и последний элементы (новые элементы добавлять после существующих элементов с такими же значениями) и вывести ссылку на последний элемент преобразованного списка.

Dynamic37. Дан первый элемент A1 непустого двусвязного списка. Продублировать в списке все элементы с нечетными номерами (новые элементы добавлять перед существующими элементами с такими же значениями) и вывести ссылку на первый элемент преобразованного списка.

Dynamic38. Дан первый элемент A1 непустого двусвязного списка. Продублировать в списке все элементы с нечетными номерами (новые элементы добавлять после существующих элементов с такими же значениями) и вывести ссылку на последний элемент преобразованного списка.

Dynamic39. Дан первый элемент A1 непустого двусвязного списка. Продублировать в списке все элементы с нечетными значениями (новые элементы добавлять перед существующими элементами с такими же значениями) и вывести ссылку на первый элемент преобразованного списка.

Dynamic40. Дан первый элемент A1 непустого двусвязного списка. Продублировать в списке все элементы с нечетными значениями (новые элементы добавлять после существующих элементов с такими же значениями) и вывести ссылку на последний элемент преобразованного списка.

Dynamic41. Дана ссылка A0 на один из элементов непустого двусвязного списка. Удалить из списка данный элемент и вывести две ссылки: на элемент, предшествующий удаленному, и на элемент, следующий за удаленным (один или оба этих элемента могут отсутствовать; для отсутствующих элементов выводить null). После удаления элемента из списка вызвать для него метод Dispose.

Dynamic42. Дан первый элемент A1 двусвязного списка, содержащего не менее двух элементов. Удалить из списка все элементы с нечетными номерами и вывести ссылку на первый элемент преобразованного списка. После удаления элементов из списка вызывать для них метод Dispose.

Dynamic43. Дан первый элемент A1 непустого двусвязного списка. Удалить из списка все элементы с нечетными значениями и вывести ссылку на первый элемент преобразованного списка (если в результате удаления элементов список окажется пустым, то вывести null). После удаления элементов из списка вызывать для них метод Dispose.

Dynamic44. Дана ссылка A0 на один из элементов непустого двусвязного списка. Переместить данный элемент в конец списка и вывести ссылки на первый и последний элементы преобразованного списка. Новые объекты типа Node не создавать, свойства Data не изменять.

Dynamic45. Дана ссылка A0 на один из элементов непустого двусвязного списка. Переместить данный элемент в начало списка и вывести ссылки на первый и последний элементы преобразованного списка. Новые объекты типа Node не создавать, свойства Data не изменять.

Dynamic46. Дано число K (> 0) и ссылка A0 на один из элементов непустого двусвязного списка. Переместить в списке данный элемент на K позиций вперед (если после данного элемента находится менее K элементов, то переместить его в конец списка). Вывести ссылки на первый и последний элементы преобразованного списка. Новые объекты типа Node не создавать, свойства Data не изменять.

Dynamic47. Дано число K (> 0) и ссылка A0 на один из элементов непустого двусвязного списка. Переместить в списке данный элемент на K позиций назад (если перед данным элементом находится менее K элементов, то переместить его в начало списка). Вывести ссылки на первый и последний элементы преобразованного списка. Новые объекты типа Node не создавать, свойства Data не изменять.

Dynamic48. Даны ссылки AX и AY на два различных элемента двусвязного списка (элемент AX находится в списке перед элементом AY, но не обязательно рядом с ним). Поменять местами данные элементы и вывести ссылку на первый элемент преобразованного списка. Новые объекты типа Node не создавать, свойства Data не изменять.

Dynamic49. Дан первый элемент A1 непустого двусвязного списка. Перегруппировать элементы списка, переместив все элементы с нечетными номерами в конец списка (в том же

порядке) и вывести ссылку на первый элемент преобразованного списка. Новые объекты типа Node не создавать, свойства Data не изменять.

Dynamic50. Дан первый элемент A1 непустого двусвязного списка. Перегруппировать элементы списка, переместив все элементы с нечетными значениями в конец списка (в том же порядке) и вывести ссылку на первый элемент преобразованного списка. Новые объекты типа Node не создавать, свойства Data не изменять.

Dynamic51. Для двух непустых двусвязных списков даны следующие объекты: A1 и A2 — начало и конец первого списка, A0 — один из элементов второго списка. Объединить исходные списки, поместив все элементы первого списка (в том же порядке) перед данным элементом второго списка, и вывести ссылки на первый и последний элементы объединенного списка. Новые объекты типа Node не создавать.

Dynamic52. Для двух непустых двусвязных списков даны следующие объекты: A1 и A2 — начало и конец первого списка, A0 — один из элементов второго списка. Объединить исходные списки, поместив все элементы первого списка (в том же порядке) после данного элемента второго списка, и вывести ссылки на первый и последний элементы объединенного списка. Новые объекты типа Node не создавать.

Dynamic53. Даны ссылки AX и AY на два различных элемента двусвязного списка; элемент AX находится в списке перед элементом AY, но не обязательно рядом с ним. Переместить элементы, расположенные между данными элементами (включая данные элементы), в новый список (в том же порядке). Вывести ссылки на первые элементы преобразованного и нового списков. Если преобразованный список окажется пустым, то связанную с ним ссылку положить равной null. Новые объекты типа Node не создавать.

Dynamic54. Даны ссылки AX и AY на два различных элемента двусвязного списка; элемент AX находится в списке перед элементом AY, но не обязательно рядом с ним. Переместить элементы, расположенные между данными элементами (не включая данные элементы), в новый список (в том же порядке). Вывести ссылки на первые элементы преобразованного и нового списков. Если новый список окажется пустым, то связанную с ним ссылку положить равной null. Новые объекты типа Node не создавать.

Dynamic55. Дан первый элемент A1 непустого двусвязного списка. Преобразовать список в циклический, записав в свойство Next последнего элемента списка ссылку на его первый элемент, а в свойство Prev первого элемента — ссылку на последний элемент. Вывести ссылку на элемент, который был последним элементом исходного списка.

Dynamic56. Даны ссылки A1 и A2 на первый и последний элементы непустого двусвязного списка, содержащего четное количество элементов. Преобразовать список в два циклических списка (см. задание Dynamic55), первый из которых содержит первую половину элементов исходного списка, а второй — вторую половину. Вывести ссылки A3 и A4 на два средних элемента исходного списка (элемент A3 должен входить в первый циклический список, а элемент A4 — во второй). Новые объекты типа Node не создавать.

Dynamic57. Дано число K (> 0) и ссылки A1 и A2 на первый и последний элементы непустого двусвязного списка. Осуществить циклический сдвиг элементов списка на K позиций вперед (т. е. в направлении от начала к концу списка) и вывести ссылки на первый и последний элементы полученного списка. Для выполнения циклического сдвига преобразовать исходный список в циклический (см. задание Dynamic55), после чего «разорвать» его в позиции, соответствующей данному значению K. Новые объекты типа Node не создавать.

Dynamic58. Дано число K (> 0) и ссылки A1 и A2 на первый и последний элементы непустого двусвязного списка. Осуществить циклический сдвиг элементов списка на K позиций назад (т. е. в направлении от конца к началу списка) и вывести ссылки на первый и последний элементы полученного списка. Для выполнения циклического сдвига преобразовать исходный список в циклический (см. задание Dynamic55), после чего «разорвать» его в позиции, соответствующей данному значению K. Новые объекты типа Node не создавать.

Dynamic59. Даны ссылки A1, A2 и A3 на первый, последний и текущий элементы двусвязного списка (если список является пустым, то $A1 = A2 = A3 = \text{null}$). Также дано число $N (> 0)$ и набор из N чисел. Описать класс IntList, содержащий следующие члены:

- три закрытых поля first, last, current типа Node (первый, последний и текущий элементы списка);
- конструктор с параметрами aFirst, aLast, aCurrent — первым, последним и текущим элементами существующего списка;
- процедура InsertLast(D), которая добавляет новый элемент со значением D в конец списка (D — входной параметр целого типа, добавленный элемент становится текущим);
- процедура Put (без параметров), которая выводит ссылки на поля first, last и current, используя метод Put класса PT.

С помощью метода InsertLast добавить в конец исходного списка данный набор чисел (в том же порядке) и вывести ссылки на первый, последний и текущий элементы полученного списка, используя для этого метод Put класса IntList.

Dynamic60. Даны ссылки A1, A2 и A3 на первый, последний и текущий элементы двусвязного списка (если список является пустым, то $A1 = A2 = A3 = \text{null}$). Также дано число $N (> 0)$ и набор из N чисел. Включить в класс IntList (см. задание Dynamic59) процедуру InsertFirst(D), которая добавляет новый элемент со значением D в начало списка (D — входной параметр целого типа). Добавленный элемент становится текущим. С помощью метода InsertFirst добавить в начало исходного списка данный набор чисел (добавленные числа будут располагаться в списке в обратном порядке) и вывести ссылки на первый, последний и текущий элементы полученного списка.

Dynamic61. Даны ссылки A1, A2 и A3 на первый, последний и текущий элементы непустого двусвязного списка. Также даны пять чисел. Включить в класс IntList (см. задание Dynamic59) процедуру InsertBefore(D), которая вставляет новый элемент со значением D перед текущим элементом списка (D — входной параметр целого типа). Вставленный элемент становится текущим. С помощью метода InsertBefore вставить пять данных чисел в исходный список и вывести ссылки на первый, последний и текущий элементы полученного списка.

Dynamic62. Даны ссылки A1, A2 и A3 на первый, последний и текущий элементы непустого двусвязного списка. Также даны пять чисел. Включить в класс IntList (см. задание Dynamic59) процедуру InsertAfter(D), которая вставляет новый элемент со значением D после текущего элемента списка (D — входной параметр целого типа). Вставленный элемент становится текущим. С помощью метода InsertAfter вставить пять данных чисел в исходный список и вывести ссылки на первый, последний и текущий элементы полученного списка.

Dynamic63. Даны ссылки A1, A2 и A3 на первый, последний и текущий элементы непустого двусвязного списка. Включить в класс IntList (см. задание Dynamic59) процедуры ToFirst (делает текущим первый элемент списка), ToNext (делает текущим в списке следующий элемент, если он существует), SetData(D) (присваивает текущему элементу списка значение D целого типа), а также функцию IsLast логического типа (возвращает true, если текущий элемент списка является его последним элементом, и false в противном случае). Методы ToFirst, ToNext и IsLast не имеют параметров; параметр D метода SetData является входным параметром целого типа. С помощью данных методов присвоить нулевые значения элементам исходного списка с нечетными номерами и вывести количество элементов в списке, а также ссылки на первый, последний и текущий элементы преобразованного списка (текущим элементом должен стать последний элемент списка).

Dynamic64. Даны ссылки A1, A2 и A3 на первый, последний и текущий элементы непустого двусвязного списка. Включить в класс IntList (см. задание Dynamic59) процедуры ToLast (делает текущим последний элемент списка), ToPrev (делает текущим в списке предыдущий элемент, если он существует) и функции GetData целого типа (возвращает значение текущего элемента списка), IsFirst логического типа (возвращает true, если текущий элемент списка является его первым элементом, и false в противном случае). Данные методы не имеют

параметров. С помощью этих методов вывести все четные значения элементов исходного списка, просматривая список с конца. Вывести также количество элементов в списке.

Dynamic65. Даны ссылки A1, A2 и A3 на первый, последний и текущий элементы двусвязного списка, содержащего не менее пяти элементов. Включить в класс `IntList` (см. задание Dynamic59) функцию `DeleteCurrent` целого типа (без параметров), удаляющую из списка текущий элемент и возвращающую его значение. После удаления элемента текущим становится следующий элемент или, если следующего элемента не существует, последний элемент списка. Метод `DeleteCurrent` также вызывает для удаленного элемента метод `Dispose`. С помощью метода `DeleteCurrent` удалить из исходного списка пять элементов и вывести их значения. Вывести также ссылки на первый, последний и текущий элементы преобразованного списка (для пустого списка вывести три константы `null`).

Dynamic66. Даны ссылки A1, A2 и A3 на первый, последний и текущий элементы непустого двусвязного списка. Включить в класс `IntList` (см. задание Dynamic59) классический метод — процедуру `Split(L1, L2)`, которая переносит элементы списка L1 от текущего до последнего в новый список L2 (таким образом, список L1 делится на две части, причем первая часть может оказаться пустой). Параметры процедуры имеют тип `IntList`; первый параметр является входным, второй — выходным. Текущими элементами непустых результирующих списков становятся их первые элементы. Новые объекты типа `Node` в процедуре `Split` не создавать. С помощью этой процедуры разбить исходный список на два и вывести ссылки на первый, последний и текущий элементы каждого из полученных списков (для пустого списка выводятся три константы `null`).

Dynamic67. Даны ссылки на первый, последний и текущий элементы двух непустых двусвязных списков. Включить в класс `IntList` (см. задание Dynamic59) классический метод — процедуру `Add(L1, L2)`, которая добавляет все элементы из списка L1 (в том же порядке) в конец списка L2; в результате список L1 становится пустым. Текущим элементом дополненного списка становится первый из добавленных элементов. Оба параметра процедуры имеют тип `IntList` и являются входными. Новые объекты типа `Node` в процедуре `Add` не создавать. С помощью этой процедуры добавить первый из данных списков в конец второго и вывести ссылки на первый, последний и текущий элементы объединенного списка.

Dynamic68. Даны ссылки на первый, последний и текущий элементы двух непустых двусвязных списков. Включить в класс `IntList` (см. задание Dynamic59) классический метод — процедуру `Insert(L1, L2)`, которая вставляет все элементы из списка L1 (в том же порядке) в список L2 перед его текущим элементом; в результате список L1 становится пустым. Текущим элементом списка L2 становится первый из вставленных элементов. Оба параметра процедуры имеют тип `IntList` и являются входными. Новые объекты типа `Node` в процедуре `Insert` не создавать. С помощью этой процедуры вставить первый из данных списков в текущую позицию второго и вывести ссылки на первый, последний и текущий элементы объединенного списка.

Dynamic69. Даны ссылки на первый, последний и текущий элементы двух двусвязных списков (второй список может быть пустым). Включить в класс `IntList` (см. задание Dynamic59) классический метод — процедуру `MoveCurrent(L1, L2)`, которая перемещает текущий элемент списка L1 в список L2 (элемент вставляется после текущего элемента списка L2 и сам становится текущим; в списке L1 текущим становится следующий элемент или, если следующего элемента не существует, последний элемент). Оба параметра процедуры имеют тип `IntList` и являются входными. Новые объекты типа `Node` в процедуре `MoveCurrent` не создавать. С помощью этой процедуры переместить текущий элемент первого списка во второй и вывести ссылки на первый, последний и текущий элементы каждого из полученных списков (для пустого списка выводятся три константы `null`).

ДИНАМИЧЕСКИЕ СТРУКТУРЫ ДАННЫХ: СПИСОК С БАРЬЕРНЫМ ЭЛЕМЕНТОМ

Реализация двусвязного списка в виде цепочки узлов, ограниченной по краям нулевыми ссылками `null`, не является единственно возможной. Двусвязный список можно также реализовать в виде замкнутой цепочки узлов с дополнительным фиктивным, или барьерным, элементом. Этот барьерный элемент связан своими свойствами `Next` и `Prev` с первым и последним «настоящим» элементом списка соответственно, поэтому, имея ссылку на барьерный элемент, можно сразу перейти как к первому, так и к последнему элементу списка (естественно, первый и последний элементы также связаны с барьерным элементом своими свойствами `Prev` и `Next` соответственно). Для работы с двусвязным списком, снабженным барьерным элементом, достаточно хранить две ссылки: `Barrier`, указывающую на барьерный элемент, и `Current`, указывающую на текущий элемент (который может быть как «настоящим», так и барьерным элементом). Свойство `Data` барьерного элемента может быть произвольным; для определенности его можно положить равным 0. Пустой список в данной реализации представляет собой единственный барьерный элемент, «замкнутый на себя».

Dynamic70. Даны ссылки `A1` и `A2` на первый и последний элементы двусвязного списка, реализованного в виде цепочки узлов, которая ограничена по краям константами `null` (если список пуст, то $A1 = A2 = \text{null}$). Преобразовать исходный список в циклический список (см. задание `Dynamic55`), снабженный барьерным элементом. Барьерный элемент должен иметь значение 0 и быть связан своими свойствами `Next` и `Prev` с первым и последним элементом исходного списка (в случае пустого исходного списка свойства `Next` и `Prev` барьерного элемента должны указывать на сам барьерный элемент). Вывести ссылку на барьерный элемент полученного списка. Не создавать новые объекты типа `Node`, за исключением барьерного элемента.

Dynamic71. Даны ссылки `A1` и `A2` на барьерный и текущий элементы двусвязного списка (о списке с барьерным элементом см. задание `Dynamic70`). Разбить список на два, перенеся во второй список все элементы от текущего до последнего и добавив ко второму списку барьерный элемент. Если текущий элемент исходного списка является барьерным элементом, то второй список должен быть пустым (т. е. состоять только из барьерного элемента). Вывести ссылку на барьерный элемент второго списка. Не создавать новые объекты типа `Node`, за исключением барьерного элемента для второго списка.

Dynamic72. Даны ссылки `A1` и `A2` на барьерные элементы двух двусвязных списков (о списке с барьерным элементом см. задание `Dynamic70`). Объединить исходные списки, связав конец первого и начало второго списка (барьерным элементом объединенного списка должен остаться барьерный элемент первого списка). Вывести ссылки на первый и последний элементы объединенного списка (если объединенный список является пустым, то дважды вывести ссылку на его барьерный элемент). После удаления лишнего барьерного элемента вызвать для него метод `Dispose`.

Dynamic73. Даны ссылки `A1` и `A2` на барьерные элементы двух двусвязных списков (о списке с барьерным элементом см. задание `Dynamic70`). Объединить исходные списки, связав конец первого и начало второго списка (барьерным элементом объединенного списка должен остаться барьерный элемент второго списка). Вывести ссылки на первый и последний элементы объединенного списка (если объединенный список является пустым, то дважды вывести ссылку на его барьерный элемент). После удаления лишнего барьерного элемента вызвать для него метод `Dispose`.

Dynamic74. Даны ссылки `A1` и `A2` на барьерный и текущий элементы двусвязного списка (о списке с барьерным элементом см. задание `Dynamic70`). Также дано число $N (> 0)$ и набор из N чисел. Описать класс `IntListB`, содержащий следующие члены:

- закрытые поля `barrier` и `current` типа `Node` (барьерный и текущий элементы списка);
- конструктор с параметрами `aBarrier` и `aCurrent` — барьерным и текущим элементами существующего списка;

- процедура InsertLast(D), которая добавляет новый элемент со значением D в конец списка (D — входной параметр целого типа, добавленный элемент становится текущим);
- процедура Put (без параметров), которая выводит ссылку на поле current, используя метод Put класса PT.

С помощью метода InsertLast добавить в конец исходного списка данный набор чисел (в том же порядке) и вывести ссылку на текущий элемент полученного списка, используя для этого метод Put класса IntListB.

Dynamic75. Даны ссылки A1 и A2 на барьерный и текущий элементы двусвязного списка. Также дано число N (> 0) и набор из N чисел. Включить в класс IntListB (см. задание Dynamic74) процедуру InsertFirst(D), которая добавляет новый элемент со значением D в начало списка (D — входной параметр целого типа). Добавленный элемент становится текущим. С помощью метода InsertFirst добавить в начало исходного списка данный набор чисел (добавленные числа будут располагаться в списке в обратном порядке) и вывести ссылку на текущий элемент полученного списка.

Dynamic76. Даны ссылки A1 и A2 на барьерный и текущий элементы двусвязного списка. Также даны пять чисел. Включить в класс IntListB (см. задание Dynamic74) процедуру InsertBefore(D), которая вставляет новый элемент со значением D перед текущим элементом списка (D — входной параметр целого типа). Вставленный элемент становится текущим. С помощью метода InsertBefore вставить пять данных чисел в исходный список и вывести ссылку на текущий элемент полученного списка.

Dynamic77. Даны ссылки A1 и A2 на барьерный и текущий элементы двусвязного списка. Также даны пять чисел. Включить в класс IntListB (см. задание Dynamic74) процедуру InsertAfter(D), которая вставляет новый элемент со значением D после текущего элемента списка (D — входной параметр целого типа). Вставленный элемент становится текущим. С помощью метода InsertAfter вставить пять данных чисел в исходный список и вывести ссылку на текущий элемент полученного списка.

Dynamic78. Даны ссылки A1 и A2 на барьерный и текущий элементы двусвязного списка. Включить в класс IntListB (см. задание Dynamic74) процедуры ToFirst (делает текущим первый элемент списка), ToNext (делает текущим следующий элемент в списке), SetData(D) (присваивает текущему элементу списка значение D целого типа, если данный элемент не является барьерным) и функцию IsBarrier логического типа (возвращает true, если текущий элемент списка является его барьерным элементом, и false в противном случае). Методы ToFirst, ToNext и IsBarrier не имеют параметров. Параметр D метода SetData является входным параметром целого типа. С помощью этих методов присвоить нулевые значения элементам исходного списка с нечетными номерами и вывести количество элементов в списке, а также ссылку на новый текущий элемент списка (текущим элементом списка должен стать его барьерный элемент). Нумерация ведется от первого элемента списка; барьерный элемент не нумеруется и не учитывается при подсчете элементов.

Dynamic79. Даны ссылки A1 и A2 на барьерный и текущий элементы двусвязного списка. Включить в класс IntListB (см. задание Dynamic74) процедуры ToLast (делает текущим последний элемент списка), ToPrev (делает текущим предыдущий элемент в списке) и функцию GetData целого типа (возвращает значение текущего элемента списка L). Данные методы не имеют параметров. С помощью этих методов, а также функции IsBarrier из задания Dynamic78, вывести все четные значения элементов исходного списка, просматривая список с конца. Вывести также количество элементов в списке. Барьерный элемент не обрабатывается и не учитывается при подсчете элементов.

Dynamic80. Даны ссылки A1 и A2 на барьерный и текущий элементы непустого двусвязного списка, причем текущий элемент не совпадает с барьерным. Включить в класс IntListB (см. задание Dynamic74) функцию DeleteCurrent целого типа, удаляющую из списка текущий элемент и возвращающую его значение. Текущим становится следующий элемент или, если следующий элемент является барьерным, предыдущий элемент списка. Функция также вызывает для удаленного элемента метод Dispose. Если текущим элементом является

барьерный элемент, то функция не выполняет никаких действий и возвращает 0. С помощью этой функции, а также метода `IsBarrier` из задания `Dynamic78`, удалить из исходного списка пять элементов (или все элементы, если их менее пяти) и вывести их значения. Вывести также ссылку на новый текущий элемент списка.